

Esercitazione

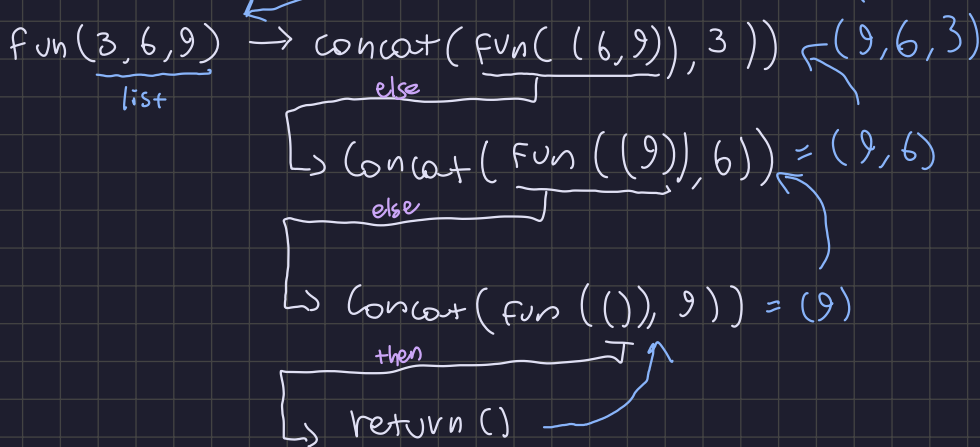
Trasformare una procedura ricorsiva data in una procedura ricorsiva in coda.

I passi da eseguire sono:

- Definizione breve di ricorsione
- Definizione breve di ricorsione in coda
- Esecuzione della procedura data sull'input specificato
- Breve descrizione a parole di cosa fa il programma
- Scrittura della procedura ricorsiva in coda (con chiamata iniziale e inizializzazione di res)
- Esecuzione della procedura scritta sull'input specifico

```
1 lista fun(lista list) {  
  if length(list) = 0 return list;  
  else return concat(fun(cdr(list), car(list)));  
  // crd è la coda della lista (restituisce una lista)  
  // car è la testa (restituisce un elemento)  
}
```

Eseguire su (3,6,9):

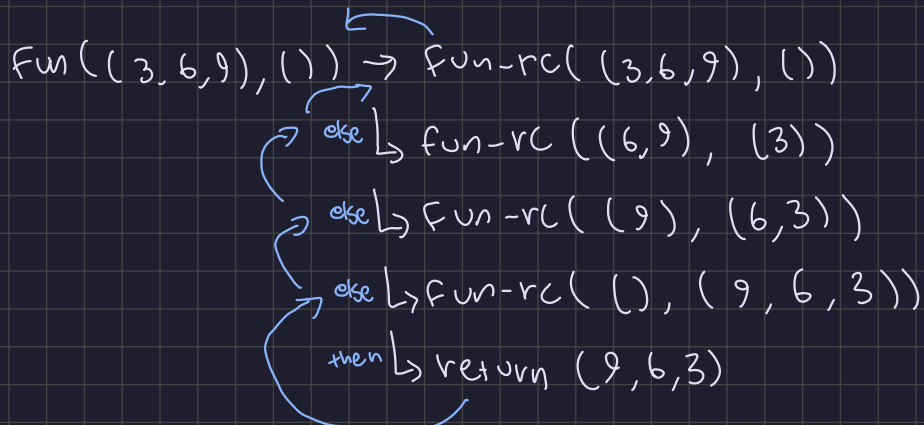


Il programma esegue una reverse della lista.

Fornire la versione ricorsiva in coda:

```
int fun(lista list) {  
  fun-rc(list, ());  
}  
  
fun-rc(lista list, list res) {  
  if length(list) = 0 return res;  
  else res = fun-rc(cdr(list), concat(car(res), res));  
}
```

Eseguiamo su (3,6,9)



```

2 int fun(lista list) {
    if length(list) = 0 return 0;
    else return (car(list) + fun(cdr(list)));
}

```

Esecuzione su (3,6,9)

$Fun((3,6,9)) \rightarrow 3 + Fun((6,9)) = 18$
 else $\rightarrow 6 + Fun((9)) = 15$
 else $\rightarrow 9 + Fun(()) = 9$
 then $\rightarrow return 0$

La funzione calcola la somma degli elementi che compongono la lista

Conversione in coda

```

int fun(lista list) {
    fun-rc(list, 0);
}

int fun-rc(lista list, int res) {
    if length(list) = 0 return res;
    else return fun-rc(cdr(list), res + car(list));
}

```

Eseguiamo su (3,6,9)

$Fun((3,6,9)) \rightarrow Fun-rc((3,6,9), 0)$
 else $\rightarrow Fun-rc((6,9), 0 + 3)$
 else $\rightarrow Fun-rc((9), 0 + 3 + 6)$
 else $\rightarrow Fun-rc((), 0 + 3 + 6 + 9)$
 then $\rightarrow return 18$

```

3 lista fun(lista list) {
    if length(list) = 0 return ();
    else return concat(2*car(list), fun(crd(list)));
}

```

Eseguire su (3,6,9)

$Fun(3,6,9) \rightarrow concat(2 \cdot 3, Fun((6,9))) = concat(6, (12, 18)) = (6, 12, 18)$
 $\rightarrow concat(2 \cdot 6, Fun((9))) = concat(12, (18)) = (12, 18)$
 $\rightarrow concat(2 \cdot 9, Fun(())) = concat(18, ()) = (18)$



La funzione moltiplica per due tutti gli elementi della lista senza modificare l'ordine

Versione in coda

```
lista fun(lista list) {  
  return fun-rc(list, ());  
}
```

```
lista fun-rc(lista list, lista res) {  
  if length(list) == 0 return res;  
  else fun-rc(cdr(list), concat(res, 2 * car(res)));  
}
```

$Fun(3, 6, 9) \rightarrow Fun-rc(3, 6, 9, (1))$

$\rightarrow Fun-rc(6, 9, concat(1, 6) = (6))$

$\rightarrow Fun-rc(9, concat(6, 12) = (6, 12))$

$\rightarrow Fun-rc((), concat(6, 12, 18) = (6, 12, 18))$